Fase 1: Construcción del Prototipo & Pruebas en Local

1. Ejecuta el Prompt Maestro ➔ 2. Entrena en tu terminal con `npm run dev` ➔ 3. Valida en `localhost:3000`.

🤖 1. PROMPT MAESTRO PARA ANTIGRAVITY (Creación Integral de app/page.js)

Copiar y Pegar en el Chat

```
"Actúa como un arquitecto frontend y fullstack senior de Next.js (App Router).
Construye en app/page.js la interfaz completa, moderna y 100% responsiva para el cotizador inteligente 'VibeCargo':

1. DISEÑO & ESTILO (Tailwind CSS):
   - Hero header con gradiente moderno, badge animado ('Cotizador IA en Vivo') y diseño limpio con soporte para modo oscuro/claro.
   - Layout en 2 columnas en desktop (Formulario a la izquierda y Tarjeta de Resultados/Desglose a la derecha).

2. FORMULARIO REACTIVO (Campos de Entrada):
   - Dirección de Origen (input de texto con icono de mapa).
   - Dirección de Destino (input de texto con icono de destino).
   - Pisos por escalera (contador interactivo con botones +/- y visualizador numérico).
   - Descripción de Muebles / Ítems (textarea con placeholders sugeridos).
   - Nombre del Cliente y Teléfono (opcionales).
   - Botón de Envío con estados visuales: Idle ('Calcular Cotización con IA 🚀'), Loading ('Calculando costos con Gemini... ⏳') y Disabled.

3. CONEXIÓN BACKEND (Llamada Asíncrona):
   - Realiza un fetch POST a '/api/quote' enviando los datos del formulario en JSON.
   - Maneja errores de conexión con alertas visuales amigables y estados de carga animados (loading skeletons).

4. TARJETA DE RESULTADOS & ACCIÓN WHATSAPP:
   - Renderiza la tabla/desglose de costos con cada ítem de 'data.breakdown' formateado en pesos chilenos ($XX.XXX CLP).
   - Muestra el precio final destacado en grande ('data.estimated_total').
   - Agrega un botón verde destacado 'Compartir Cotización por WhatsApp' que abra automáticamente la API de WhatsApp ('https://wa.me/?text=...') con el texto
codificado de 'data.whatsapp_message'."
```

💻 2. Entrenamiento en Consola & Pruebas en Vivo (Localhost)

Acción en Terminal	Comando Exacto	Qué probar en el navegador
1. Iniciar Servidor	`npm run dev` (o `bun dev`)	Abre `http://localhost:3000` en tu navegador web.
2. Probar Cotizador	Interactuar en el formulario web	Ingresa direcciones y pisos ➔ Clic en **Cotizar** ➔ Verifica que Gemini calcula el desglose.
3. Probar WhatsApp	Clic en **"Copiar a WhatsApp"**	Verifica que se abre WhatsApp con el mensaje prellenado y el total en CLP.
4. Refactor en Vivo	Pídele ajustes a Antigravity	Ej: *"Cambia el color del botón a verde y pon formato de moneda chilena con puntos."*

🗄️ 3. Script SQL en Supabase (Tabla quotes con Row Level Security)

```sql
-- 1. En Supabase: Menú izquierdo → Clic en "SQL Editor" → Clic en botón "New Query"
-- ⚠️ ATENCIÓN ALUMNO: Si tu proyecto no es de fletes, cambia "quotes" por el nombre de tu tabla:
CREATE TABLE IF NOT EXISTS public.quotes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  created_at TIMESTAMPTZ DEFAULT NOW(),
  origin_address TEXT NOT NULL,
  destination_address TEXT NOT NULL,
  floors_stairs INT DEFAULT 0,
  items_description TEXT NOT NULL,
  estimated_total NUMERIC(10,2) NOT NULL,
  breakdown_json JSONB NOT NULL,
  whatsapp_message TEXT NOT NULL,
  status TEXT DEFAULT 'pending'
);

-- 2. Habilitar Seguridad RLS y Políticas de Acceso Público:
ALTER TABLE public.quotes ENABLE ROW LEVEL SECURITY;
CREATE POLICY "permitir_insercion_publica" ON public.quotes FOR INSERT TO anon WITH CHECK (true);
CREATE POLICY "permitir_lectura_publica" ON public.quotes FOR SELECT TO anon USING (true);
-- 3. Clic en el botón verde "RUN" (o presiona Ctrl + Enter)
```

✅ **¡Tu MVP ya funciona en tu máquina!** Una vez que probaste el cotizador en `http://localhost:3000` y te encanta cómo quedó, avanzamos a la **Fase 2** (página siguiente) para vincular con SSH tu propio repositorio en GitHub y desplegar en Vercel.

© Mujica AI Lab SpA - Todos los derechos reservados. No comparta, copie, reproduzca ni venda ninguna parte de este documento a menos que tenga permiso por escrito de Mujica AI Lab SpA. Todas las infracciones serán perseguidas. Si usted es el propietario personal de Mujica AI Lab SpA con licencia de usuario final, podrá utilizarla para su propio uso, pero no para ningún otro propósito.

Fase 2: Crear tu Propio Repo en GitHub (SSH) & Despliegue Vercel

1. Crea tu repo personal con SSH ➔ 2. Despliega en Vercel ➔ 3. Activa el Keep-Alive en cron-job.org.

🐙 4. Cómo Crear y Subir a tu Propio Repositorio en GitHub (Vía SSH)

Paso en GitHub / Web	Comandos en la Terminal de VS Code / Antigravity (SSH)	Propósito
1. Crear Repo Vacío	Entrar a `https://github.com/new` ➔ Escribir nombre (ej: `mi-mvp-cotizador`) ➔ Elegir **Public** (sin marcar README) ➔ Create repository	Crea tu repositorio personal en GitHub.
2. Re-vincular por SSH	`git remote set-url origin git@github.com: TU_USUARIO / TU_REPO .git`	Apunta tu proyecto local hacia tu nuevo repo vía clave SSH.
3. Confirmar y Push	`git add .` ➔ `git commit -m "feat: mi primer MVP cotizador"` ➔ `git push -u origin main`	Sube todo el código a tu cuenta personal de GitHub.

🚀 5. Importar tu Repositorio en Vercel & Cargar Variables

Variable en Vercel	Tipo	Ruta Literal en Paneles	Dónde Pegar en Vercel
`GEMINI_API_KEY`	🔒 Servidor	Google AI Studio ➔ Get API key ➔ Create Key	Vercel ➔ Settings ➔ Environment Variables
`NEXT_PUBLIC_SUPABASE_URL`	🌐 Pública	Supabase ➔ Project Settings ➔ API ➔ **Project URL**	Vercel ➔ Settings ➔ Environment Variables
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	🌐 Pública	Supabase ➔ Project Settings ➔ API ➔ **anon public**	Vercel ➔ Settings ➔ Environment Variables

⏱️ 6. Endpoint Keep-Alive (/app/api/keep-alive/route.ts)

```ts
// app/api/keep-alive/route.ts
import { NextResponse } from "next/server";
import { getSupabaseClient } from "@/lib/supabase";

export const dynamic = "force-dynamic";
export const revalidate = 0;

export async function GET(request: Request) {
  const startTime = Date.now();
  try {
    const supabase = getSupabaseClient();
    // ⚠️ ATENCIÓN ALUMNO: Reemplaza "tu_tabla" por el nombre de tu tabla (ej: "quotes", "leads")
    const { data, error } = await supabase.from("tu_tabla").select("id").limit(1);

    // Salvavidas: Si olvidas cambiar "tu_tabla", el error 42P01 indica que Postgres respondió y está activo
    const isAlive = !error || error.code === "42P01";
    return NextResponse.json({
      ok: isAlive, status: "alive", latency_ms: Date.now() - startTime, timestamp: new Date().toISOString()
    }, { headers: { "Cache-Control": "no-store, no-cache, max-age=0" } });
  } catch (err: any) {
    return NextResponse.json({ ok: false, error: err.message }, { status: 500 });
  }
}
```

🌐 7. Automatización cron-job.org (Persistencia cada 3 días)

Paso	Botón / Pantalla Literal	Instrucción Exacta para el Alumno
Paso 1	Abrir `https://cron-job.org` ➔ Sign Up	Escribir Email y Contraseña ➔ Clic en Create Account ➔ Confirmar correo ➔ Clic en Log In.
Paso 2	Menú superior ➔ Cronjobs ➔ CREATE CRONJOB	Escribir Title: `Vibecode Supabase Keep Alive`.
Paso 3	Campo **URL**	Escribir: `https:// tu-proyecto.vercel.app /api/keep-alive`
Paso 4	Schedule & Notificaciones ➔ Botón final	Frecuencia: `Every 3 days (04:00 UTC)` ➔ Casilla Notify on failure ➔ Clic en CREATE.

© Mujica AI Lab SpA - Todos los derechos reservados. No comparta, copie, reproduzca ni venda ninguna parte de este documento a menos que tenga permiso por escrito de Mujica AI Lab SpA. Todas las infracciones serán perseguidas. Si usted es el propietario personal de Mujica AI Lab SpA con licencia de usuario final, podrá utilizarla para su propio uso, pero no para ningún otro propósito.